Operating Systems — Lab Assignment 6

Deadlock avoidance, detection, prevention, and synchronization

Name: Raj Pratap Singh Gurjar
Roll No.: 2401MC21

Overview

This lab applies four standard operating-system techniques to small, reproducible xv6 programs. The first two programs are deterministic simulations. The last two create real xv6 child processes and use pipe-backed counting semaphores so that resource ownership is visible in the terminal.

Q1 — Banker's Algorithm

bankers.c uses five processes and three resource types. It stores Allocation, Max, Available, and computes Need = Max − Allocation. The safety algorithm repeatedly finds a process whose remaining Need fits in Work, simulates its completion, and records the process in a safe sequence. The built-in P0 request [0, 0, 2] passes the basic bounds checks but makes the state unsafe, so it is rolled back. The later P1 request [1, 0, 2] is granted and the safe sequence is printed again.

Q2 — Wait-for Graph Detection

deadlockdetect.c starts with allocation and request matrices. For every resource requested by Pi, it adds an edge Pi → Pj for each process Pj holding that resource. DFS uses the usual white, gray, and black colours. A back edge to a gray vertex identifies a cycle. The second scenario intentionally contains P0 → P1 → P2 → P0, and the complete cycle is printed.

Q3 — Resource Ordering

resourceorder.c uses pipe tokens as counting-semaphore permits. In the first phase, Process A takes Lock1 before Lock2 while Process B takes Lock2 before Lock1. A delay makes the circular wait visible; the parent reports a timeout and terminates the demonstration. In the second phase, both children always take Lock1 before Lock2. This removes circular wait because a process can never wait for a lower-ranked lock while holding a higher-ranked one.

Q4 — Combined Synchronization

syncdeadlock.c creates five children and three resource pools: two Printers, one Scanner, and two Disks. Each child performs three cycles using either Printer + Scanner or Scanner + Disk. The explicit prevention rule is the global order Printer < Scanner < Disk. Every acquisition follows that order, while the pipe tokens enforce the number of available instances. Each request, grant, work interval, release, and final completion is printed.

Build and Run

Copy the four C files to xv6-public, apply the four qN Makefile patches in diffs/, and run make clean && make qemu-nox. At the xv6 shell run bankers, deadlockdetect, resourceorder, and syncdeadlock. The tests/ directory contains one test command per question. Screenshots should be captured from the xv6 terminal and saved in screenshots/.

Expected observations

Question	Expected result
Q1	Safe sequence, one denied request, one granted request
Q2	Acyclic graph followed by a printed three-process cycle

Q3	Bad-order timeout followed by fixed-order completion
Q4	All five children complete without deadlock or pool overflow

Note: Q3 intentionally uses a timeout for the bad-order case; without it, the demonstration would leave the xv6 shell blocked forever. The local host tests verify the deterministic simulations and source structure, while the process-level evidence belongs to the xv6 run.